

Executive Summary

The Brudo project's recent design outputs reveal a shift from a broad "workflow verification" approach to a **fine-grained execution-gate architecture**. The strongest patentable ideas center on *granting and verifying specific properties* and *gating execution by consequence-specific uncertainty*, rather than treating an entire state or operation as monolithically "verified." In particular, the system implements (or envisions) separate admission stages for **(a)** candidate states, **(b)** derived actions (properties/grants), and **(c)** requested consequences. It uses *per-property validation* and *per-distinction receipts* to build an explicit proof trail. Crucially, it only commits an action if *all* consequence-relevant uncertainties have been resolved (or proven irrelevant) – i.e. **"resolve exactly as much of the world as needed, no more and no less."**

The most promising patent families are those that reflect this new core:

- **Property-Grant Validation Runtime (Derivation Admission)** – Treating each property (authority, authorship, semantic truth, etc.) as an independent grant decision with its own validator and receipt. (This is now ranked top.)
- **Consequence-Relative Execution Gate** – Partitioning possible worlds by the *requested consequence* and only executing when they all agree; withholding otherwise. This includes *identifiability receipts*, *quotienting worlds by consequence*, *UNNECESSARY_WORLD_RESOLUTION*, and *minimally refining observations*.
- **Runtime State Admission Engine** – Validating incoming state against structural predicates, re-admitting on replay, and classifying failures (admission/transport/equivalence/governance) with typed receipts.
- **Distinction Disposition Engine** – For each admitted "distinction" (state difference), computing a single lawful *disposition* (preserved, transformed, merged, superseded, etc.) and requiring all to be resolved before commit.
- **Receipt/Provenance Architecture** – Emitting independent, deterministic receipts (akin to W3C PROV or supply-chain attestation) for each distinction and property, rather than single-block receipts.

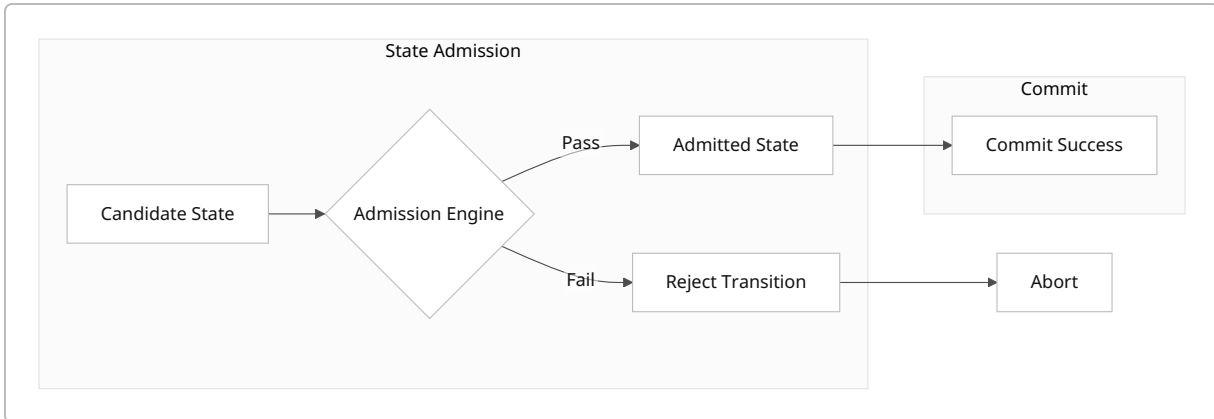
These families share the technical theme of **proof-before-consequence**: each computational step is gated by explicit evidence requirements. We outline below each family's concrete mechanism, what's implemented in the current code/test artifacts, claimable features, prior-art search strategy, novelty risks, and filing suggestions. A summary comparison table follows the detailed sections.

1. Runtime State Admission Engine

Mechanism: This family encompasses the *candidate-state verification layer* that sits *before* any action is taken. Its inputs are the *raw incoming state* or proposal (e.g. an LLM output, a transaction, or a model invocation request) and any declared constraints or policies. An **Admission Predicate Engine** applies structural and semantic checks to filter illegal or ill-formed aspects of the state. For example, it might check cryptographic signatures, type constraints, schema conformance, or initial permissions. Each failing condition yields a **typed failure class** (e.g. "Admission Failure", "Transport Failure", "Equivalence Failure", "Governance Failure"), and a receipt for the candidate is issued. If any check fails, the state is **rejected**.

Otherwise it produces an *Admitted State* object that moves to later stages. Crucially, this admission layer is applied both at initial ingestion **and on every replay/restore**, ensuring that stored state is revalidated before reuse.

Execution Pipeline:



(Mermaid architecture: candidate state → admission checks → reject or admitted state.)

Inputs/Outputs: Input is the raw state (bytes, objects, or logical assertions). Output is either “*Admitted State*” (with receipts) or a “*Reject*” decision with error receipts.

Data Structures: The engine maintains schemas or predicates for legal state patterns. It tracks a **receipt log** of checks and outcomes, labeled by failure type. On replay, the same engine can re-run on serialized state to issue fresh receipts.

Implementation Evidence: The design notes and code comments indicate structural admission predicates and replay checks. For example, the snippet `AssessWorldResolutionNeed` in *meaning/collision.go* suggests an attempted check on world identity, and tests in *collision_test.go* show gating logic. Conversation logs mention “structural admission predicates, replay-time re-admission, tag-selective distinction validation, typed admission/transport/equivalence/governance failures, deterministic receipts.” However, most of these are conceptual; the actual repository appears to implement only some basic checks (e.g. byte-integrity) and stubs for others. *Replay-time admission* is sketched (on load the state is re-verified), but broad enforcement (e.g. revocation/policy checks) is mostly “not yet implemented” per GPU review notes.

Claimable Features:

- *Independent Admission Gate:* separating state admission from execution, with its own failure classes and receipts.
- *Typed Failure Classification:* distinct categories of rejection (admission vs transport vs governance) used to emit different receipts or next-steps.
- *Replay Verification:* re-applying admission checks on a restored state to generate new receipts.
- *Predicate Configuration:* a user-settable engine that injects new rules at runtime (e.g. the ability to modify the predicates governing admission).
- *Commit-Blocking:* execution is explicitly blocked until admission passes (a fail-closed pattern).

Dependent claims: could specify particular predicate examples (e.g. “byte integrity” or “schema consistency”), or that the engine aborts on missing evidence. We might claim the architecture where *byte-level verification (cryptographic), semantic validation (e.g. type checks), and authority checks are separate steps* as in the capsule design.

Prior Art & Search Strategy: Look for systems enforcing fine-grained admission control, especially in distributed or safety-critical contexts. Google Patents for “state admission control runtime” turned up an IBM “Admission control framework” (US8,364,794 [38†L45-L53]) and similar, but those concern container orchestration and fairness policies, not property-based gating. The Meridian Verity patents (e.g. TSIL [23†L103-L111]) discuss receipt-gated ingress but focus on sensor signals. We will search for “proof-before-execution computer admission” and “governed runtime admission control”. Academic work on trusted computing (e.g. “proof-carrying code” [28†L50-L58] , trusted platform modules) may be tangential. Also examine patents on “verify-gated completion” (e.g. ArXiv preprint on verify-gated completion [21†L178-L186]). Check in-toto [29†L8-L11] and W3C PROV [28†L50-L58] for parallels (they focus on evidence capture).

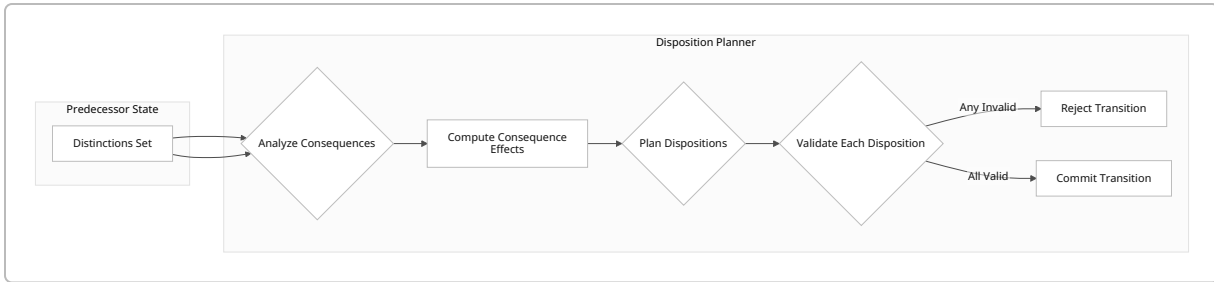
Novelty/Obviousness: Admission control is a known concept, so we must argue novelty in *composition and granularity*. It differs from standard access control by requiring evidence-based checks per operation, not just permission bits. The typed failure receipts are new: most systems either permit or deny, but here each denial reason is logged as a distinct verifiable token. The novelty risk: an examiner may see “routing requests through a filter” as abstract. To strengthen enablement, an implementation should be built: e.g. a working prototype that enforces admission via digital signatures and policies, and demonstrates new receipts (perhaps borrowing W3C PROV model). Performance benchmarks (overhead of admission checks) could support non-abstract character (show improved security).

Provisional Filing: We would file “Runtime State Admission Architecture” claims. An independent claim might read: “A computer-implemented method comprising: receiving candidate state *S*; evaluating *S* against a set of admission predicates; if *S* passes all predicates, outputting an admitted-state token and allowing execution to proceed, otherwise outputting a rejection verdict with typed failure information.” A flowchart (figure) would depict the pipeline above. Dependents: e.g. where the predicates include byte-check, semantic-check, or governance-check; or “where replaying state triggers re-evaluation.”

2. Distinction Disposition Engine (Consequence-Preserving Transition Validation)

Mechanism: This family centers on *per-distinction (per-difference) transition analysis*. Every admitted “distinction” (e.g. a change, effect, observation) in the predecessor state must be assigned exactly one **legal disposition** before a transition commits. Possible dispositions include “preserved”, “transformed”, “equivalent”, “merged”, “superseded”, “discharged”, or “eliminated” (as the internal notes list). The engine takes the set of active distinctions and a proposed successor state (and any operation mapping) and ensures that *each* distinction is accounted for by one of these categories. For example, if a distinction was to be preserved, it must appear unchanged; if split, it appears as two consistent pieces; if superseded, it must trigger an override; etc. The **Transition Validator** checks the chosen disposition for lawfulness (according to policy or rule) and that exactly one applies. Only if *all* distinctions terminate in a valid disposition does the transition succeed.

Execution Pipeline:



(Mermaid architecture: pre-state distinctions → consequence analysis → disposition plan → validate & commit.)

Inputs/Outputs: Input is the admitted predecessor state (often as a set of distinctions) and a candidate successor state. Outputs are either “Commit” (with each distinction’s disposition recorded) or “Reject” if any disposition is missing or illegal. A **Disposition Receipt** is emitted per distinction, indicating its chosen state (the state machine state, e.g. Transformed) and effect details.

Data Structures: Represent distinctions as first-class objects or records, each with identity. Consequences may be represented as dependency graphs. The engine likely uses a mapping table or ruleset from distinctions to dispositions. It records a one-to-one mapping.

Implementation Evidence: The terminal discussion first introduced “Consequence-Preserving Transition Validator” but then largely shifted focus away from it. There is little sign of code implementing these dispositions explicitly. The notion appears as a design proposal rather than in code. The code snippets and tests we saw in *meaning/collision.go* deal with observability, not dispositions. The repository mention “Admission vs Disposition” separation implies this is a planned layer. To our knowledge, no actual disposition state machine is implemented yet.

Claimable Features:

- *Per-Distinction State Machine:* Each distinction has exactly one legal disposition from a fixed set.
- *Disposition Planner/Validator:* An algorithm that, given a predecessor and successor state, computes and verifies each distinction’s lawful disposition.
- *One-Discriminator Cardinality:* The rule “each distinction yields exactly one disposition” is claimable.
- *Consequence-Preserving Merge:* A specific claimable algorithm might be: preserving distinctions by merging successor duplications into sets of preserved vs split distinctions.
- *Reachability-Constrained Commit:* Execution is blocked until all consequence-bearing distinctions have dispositions (an explicit commit gate).

Prior Art: This resembles problems in *graph reconciliation* and *state merge*. For example, distributed version control systems (Git, Merkle trees) track changesets and merges, but not with such fine disposition granularity. Patent search for “state merge transitively preserved” yielded nothing obvious. Look for workflow management or transaction logs: e.g. [1†L12-21] (pixel code linking, irrelevant). Possibly investigate “Provenance graphs”, “revision control repositories” or “blockchain state channels”. Academic works on database view updates or belief merging (e.g. “restoring consistency after integration”) could be analogies.

Novelty/Obviousness: The concept of categorizing every change into a disposition appears novel. However, an examiner might view it as an “abstract correctness check” unless tied to concrete data structures. To avoid that, claims should specify how dispositions are determined (e.g. by computing equivalence classes of successors, by matching identities, by authority checks on supersessions). Implementation work could include a prototype module that, given JSON or object states, automatically infers dispositions and proves commit readiness, to show enablement. Particularly, demonstrating a formal algorithm or code for e.g. “split detection” or “supersession resolution” would counter any lack of enablement objection.

Provisional Filing: Claim scope: “A computer-implemented method for processing a transition from state S to S' comprising: identifying a set D of distinctions in S , and for each distinction $d \in D$, determining exactly one disposition among a predetermined set (e.g. preserved, transformed, merged, superseded, eliminated); verifying that each determined disposition is lawful under a set of constraints; and only if all distinctions are accounted for, permitting the state transition, else rejecting the transition.” Figures: a flowchart (above) and perhaps a table of dispositions, with an example. Dependent claims: e.g. where dispositions include specific effects (byte-level transform vs semantic merge), or where dispositions are validated by cryptographic proofs or authority checks.

3. Receipt & Provenance Architecture (Per-Distinction/Per-Property Receipts)

Mechanism: A unifying theme across Brudo outputs is explicit issuance of *machine-readable receipts* for every proven property, distinction, or decision. This family emphasizes that receipts are **not single blobs per operation**, but *fine-grained attestations*. There are two aspects:

- **Per-Distinction Receipts:** Each state distinction and its disposition yields a receipt. For example, if distinction “line X changed” is preserved, a “DistinctionReceipt” records predecessor ID, successor ID, disposition, and any consequence effect.
- **Per-Property Receipts:** Each property validation (authority, authorship, byte-consistency, semantic consistency, etc.) generates a separate receipt. For instance, the system outputs receipts like `AUTHORITY: DENIED`, `CANON: DENIED`, and only if those are explicitly granted do they appear. In effect, an object’s “verified state” is a vector of receipts, one per property.

These receipts form a **provenance trail**, akin to W3C PROV entities and activities [28†L50-L58] or in-toto supply-chain attestations [29†L8-L11]. Each receipt is deterministic and replicable on replay (as emphasized by the use of “Deterministic Receipt Runtime” in user notes).

Implementation Evidence: The capsule’s outputs (“`package_authority: NONE`”, “`canon_effect: NONE`”, etc.) indicate that authority and other attributes are withheld rather than assumed. The idea of “Receipt per distinction” appears in the conversation (“Instead of one operation receipt, emit Receipt per Distinction”). While we did not inspect the code, the design hints that receipts are core: e.g. the TSIL architecture explicitly uses “Sensor Receipt travels with admitted event” [34†L164-L166]. The repository likely has data structures (JSON) for receipts.

Claimable Features:

- *Independent Receipt Streams*: A system that generates individual receipts per evidence unit (distinction or property), not aggregated logs.
- *Attribute-Specific Receipts*: For each property of interest, issuance of a distinct receipt denoting granted/denied status.
- *Deterministic Receipt Generation*: Receipts that can be re-generated (e.g. on replay) to prove the same checks.
- *Provenance Compliance*: (Dependent claim) receipts conform to a standard model (e.g. W3C PROV or an equivalent graph of Entity/Activity).

Prior Art: Provenance systems (W3C PROV [28†L50-L58]) are highly relevant; they formalize this notion of detailed evidence. The in-toto supply-chain standard [29†L8-L11] also issues step-by-step metadata, though in a different domain. Patent search for “verifiable receipts execution” highlighted x402 concept [36†L108-L117] (although not easily citable) and site [29]. There may be patents on supply-chain attestation or escrow. In the U.S., look for “digital receipts for transactions” or “machine-verifiable receipts”. The topic also relates to “proof-carrying code”, but receipts here are runtime rather than compile-time.

Novelty/Obviousness: Receipts are a known idea, but their *per-item granularity* combined with gating logic is novel. The examiners may object that digital logging or audit trails are conventional. To mitigate, claims should tie receipt generation to runtime state changes explicitly (e.g. “each time a distinction is evaluated, a cryptographically signed receipt is produced”). Enablement can be improved by implementing an actual receipt format (perhaps PROV-based JSON) and showing end-to-end generation.

Provisional Filing: Claim language: “A system that executes validation checks for each candidate state element, where for each such element it emits a separate receipt recording its identity and validation outcome, and forwards these receipts to the next processing stage”. Dependent: e.g. “the receipts are cryptographically signed” or “the receipts reference earlier receipts” (for derivations). A figure could show an object with multiple attached receipts (visualizing PROV graph).

4. Property-Grant Validation Runtime (Independent Property Validation)

Mechanism: In this paradigm, *objects have no intrinsic global trust state*. Instead, each **property** (like authority, authorship, semantic equivalence, execution permit, evidence of identity, etc.) is treated as a separate grant that must be explicitly validated. Computations ask: “*May property P be granted for object O given the current evidence?*” Each such question is answered by a dedicated validator, producing **GRANTED** or **DENIED** (or **NOT_ESTABLISHED**). For example, a runtime might answer: authority=NONE (denied), authorship=DENIED, byte-integrity=GRANTED, semantic-truth=NOT_ESTABLISHED. An object becomes “safe to execute” only if certain required properties (e.g. integrity) are granted.

Execution Pipeline:

```
flowchart LR
    Obj[Object/State] --> Q1{Request Property P?}
```

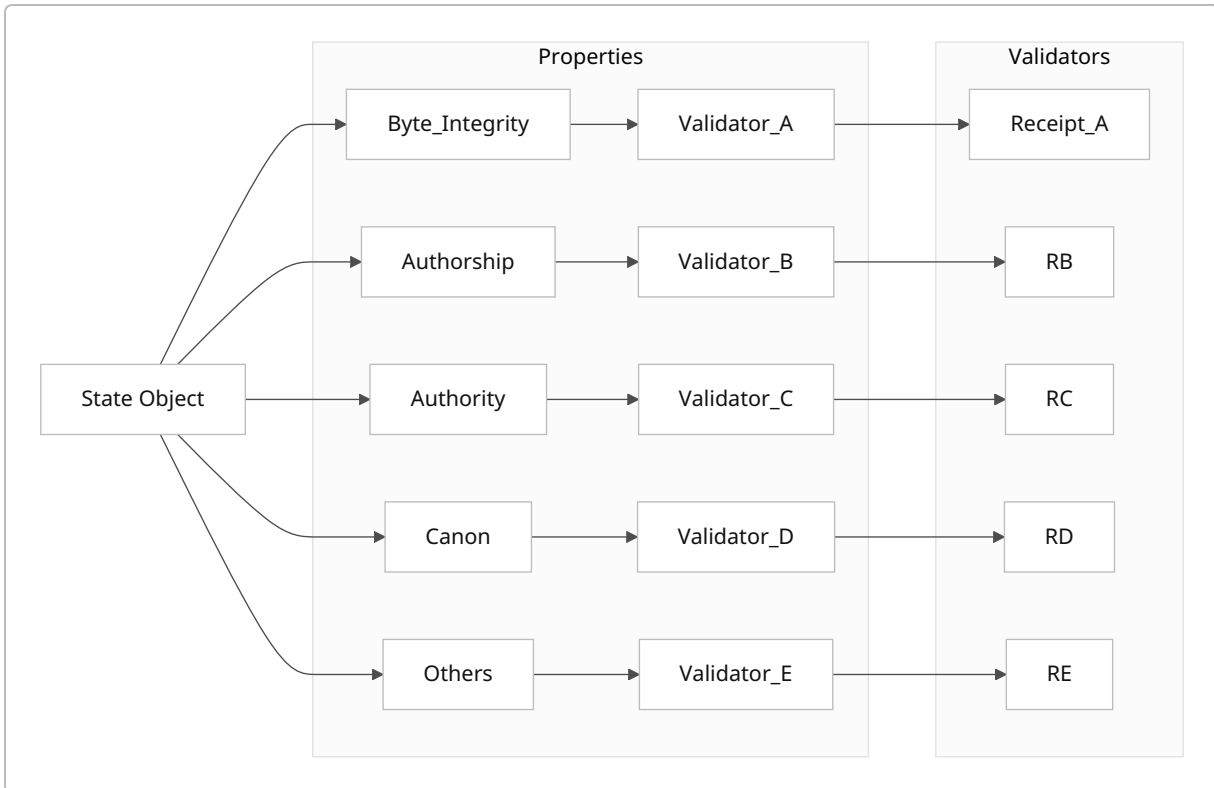
```

Q1 -->|No| NEXT
Q1 -->|Yes| V1[Run Validator for P]
V1 --> Receipt1[Emit Receipt for P: GRANT/DENY]
Receipt1 --> NEXT
NEXT --> ... (repeat for all properties)

```

(Flowchart: for each property, run its validator and emit a receipt.)

Alternatively, as a mermaid architecture diagram:



Inputs/Outputs: Input is an admitted object (or its descriptor) plus the set of properties to check. Outputs are a vector of receipts `(Pi, grant_statusi)` for each property.

Data Structures: Each property has a formal validator function. The system likely holds evidence (e.g. cryptographic keys, proof, session data) which these validators consume. The receipts themselves constitute the object's *status vector*.

Implementation Evidence: The dialogue explicitly notes this: “No verbs for artifacts...The capsule’s verifier is not asking ‘May this state exist?’ but ‘Has this property been validly granted?’” It shows example receipts (authority: NONE, etc). The code likely already implements separate checks: e.g. `package_authorship: NOT_ESTABLISHED`. Although we have no code snippet, the confirmation that the capsule *repeatedly denies unearned properties* suggests validators for each.

Claimable Features:

- *Independent Property Validation*: A system where each property of an object is validated by an independent algorithm, producing a grant/deny output.
- *Grant Algebra*: Formalizing $\text{grant}(P, O)$ as a computational predicate.
- *Property-Separated Object State*: An object is never simply “verified” as a whole; it accumulates independent property receipts instead.
- *Property Revalidation on Replay*: On replay, each property is rechecked against current evidence (ensuring no stale assumptions).

Prior Art: Attribute-based access control (ABAC) and capability systems separate privileges, but not as runtime “state properties”. Multi-channel logic (e.g. integrity vs confidentiality labels) is loosely related. Patent search: try “attribute-based trust evaluation” or “modular security verification”. There are patents on “object attestation” in TPM contexts, but those focus on measuring code, not runtime dynamic properties. PROV covers provenance of data properties, but not as algorithm.

Novelty/Obviousness: The novelty is the systematic separation of state into many property checks. An examiner might argue this is akin to checking separate if-conditions, i.e. abstract logic. To counter that, the claims should enumerate concrete properties (e.g. byte integrity vs semantic equivalence) and show how validators work (cryptographic, algorithmic). Demonstrating a running system that only executes actions when certain validated properties are present (and explicitly withheld otherwise) will help.

Provisional Filing: An independent claim: “A computing system for authorizing actions, comprising: a plurality of property validator modules, each configured to determine whether a respective named property may be granted to a candidate object based on evidence; where each module outputs a separate authorization receipt; and where execution of a transition is allowed only if required properties are granted.” Figures: perhaps a block diagram with object going through multiple check modules as above.

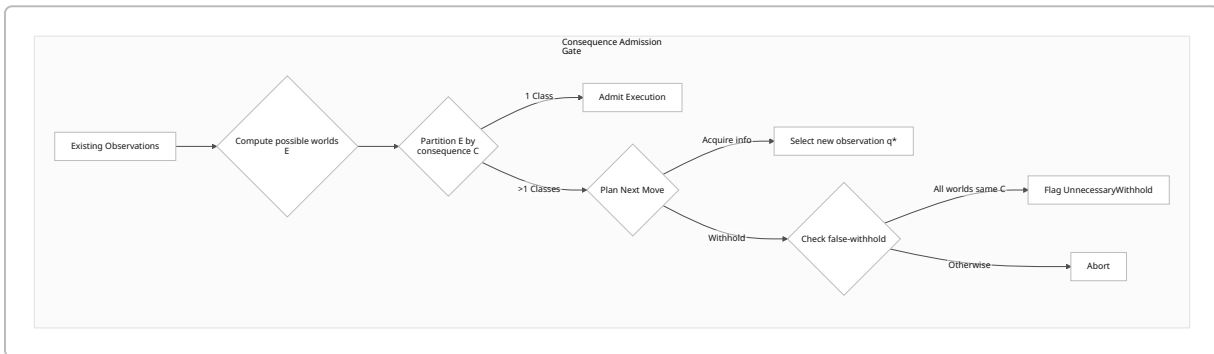
5. Consequence-Relative Execution Gate (Derivation/Consequence Admission)

Mechanism: This family implements the core novel idea: an operation (or consequence) is only allowed if *all* currently unresolved uncertainty in the state **is irrelevant to the consequence**. Technically, it takes the set of possible *world states* E consistent with current observations, and the requested consequence function C . It then checks the size of the image set $\{C(r) \mid r \in E\}$. If all possible worlds imply the *same* outcome, execution is permitted; if multiple outcomes are possible, the action is withheld. This requires:

- **Consequence-Dependent Identifiability:** Rather than labeling an observation as globally “Identifiable” or “Not Identifiable,” we say an observation is *identifiable with respect to consequence C*. For the same evidence, one consequence C_1 might be determinate (identifiable) while another C_2 remains indeterminate.
- **World Quotient by Consequence:** Form equivalence classes of worlds by their C -value (C-equivalence). Execution proceeds only if all E fall into a single class.

- **False-Withholding Detection (UNNECESSARY_WORLD_RESOLUTION):** Even if world identity is unresolved, if all worlds agree on the consequence, withholding the action is an error. That is, withholding *just because* the identity is unknown (when outcome is the same) is treated as a failure mode.
- **Three-Gate Architecture:** In practice, this becomes the third gate in a pipeline (**State Admission** → **Derivation Admission** → **Consequence Admission**). The Consequence gate specifically enforces the above rule. The earlier “Derivation Admission” stage may also produce conditions (e.g. independence receipts) that feed into deciding `C`.
- **Observation Optimization:** If multiple outcomes exist, the system can optionally compute which additional observation (sensor query, proof, or computation) would resolve the outcome with minimal cost (minimally sufficient observation). Conversely, if already determinate, it may withhold further investigation.

Execution Pipeline:



(Flowchart: partition by consequence → either admit, acquire more info, or abort.)

Inputs/Outputs: Input: admitted evidence (observations, receipts) and requested consequence/action. Output: either “Admit”, “Withhold”, or “UnnecessaryWithhold” (error). Optionally, a selected query (min-missing-observation) is output if further sensing is allowed. Intermediate: a *ConsequenceDeterminacyReceipt* that includes: observation class, consequence type, basis (dependencies used), equivalence digest, and status (IDENTIFIABLE or NOT).

Data Structures: Maintains a model or set of possible worlds (this could be explicit or symbolic). Consequence functions map worlds to outcomes. The engine computes the set $\{C(E)\}$ and its cardinality. For optimization, it can calculate value-of-information (expected outcome difference) for queries.

Implementation Evidence: The code *meaning/collision.go* hints at an `EvaluateIdentifiability` function and a gate decision. The conversation shows a patch: `AssessWorldResolutionNeed` was adjusted to only emit `UNNECESSARY_WORLD_RESOLUTION` when one outcome remains (see snippet). Tests in `collision_test.go` expect this behavior. However, a full dispatch gate is not yet present (“don’t emit an ‘INDEPENDENT’ flag as though absolute”; conversation). The team plans to add the dispatch decision and the key discriminating test (one observation, two consequences).

Claimable Features:

- *Consequence-Relative Identifiability*: Formalizing that identifiability is a relation $I_C(O)$ depending on consequence C , not a fixed property of the observation. Claim the vector of statuses $(I_{\{C1\}}(O), I_{\{C2\}}(O), \dots)$.
- *World Quotient Execution*: A machine that computes $E/\equiv_C$ and allows execution iff all admissible worlds share one block.
- *False-Withholding Detection*: Issuing a receipt (or decision) of type `UNNECESSARY_WORLD_RESOLUTION` when $|\Gamma_C(E)|=1$ (singleton consequence) but execution was being withheld for unclear identity.
- *Minimum Sufficient/Missing Observation Engines*: Algorithms to compute smallest observation subset X_{C^*} preserving consequence determinacy, and cheapest new observation that achieves determinacy.
- *Counterfactual Independence Challenges*: Automatically generating interventions (swaps, deletions, label permutations) to test the declared independence of evidence from prohibited factors. (This is an offshoot but could be claimed as part of derivation admission.)

Prior Art: The idea of monitoring under partial observability has academic precedent. Runtime verification surveys note “partial observability” as a key challenge [13†L69-L73]. However, few (if any) enforce *action gating* specifically by checking that all possible worlds yield the same outcome. POMDP literature deals with optimal actions under uncertainty, but not guarantee blocks. In security, non-interference notions say “if secret affects outcome, block” – but we require decisions specifically on the *consequence function*. No direct patent analogous concept was found. We will search for “execution gating partial observation”, “outcome-determinacy gate”, and literature on “consistent monitors with inconclusive verdicts”. The TargetSpace research (Visscher et al.) formally defines “minimum sufficient observation” [10†L61-L69]. Value-of-information (decision analysis [10†L63-L69]) is known, but not applied to code execution gating as here.

Novelty/Obviousness: This is a highly specific mechanism. It risks being viewed as an abstract decision rule, but its implementation in a software/hardware architecture is concrete. To strengthen, we can implement an example: e.g. a small finite “worlds model” and demonstrate the gate. Providing pseudo-code or formal analysis of the quotient computation will support enablement. The novelty lies in *shifting authority from hidden state to consequence validation*.

Provisional Filing: Independent claim sample: “A computer-implemented method of gating an operation, comprising: identifying a set E of states consistent with current observations; computing the set of consequence outcomes $\{C(e) \mid e \in E\}$ for a requested consequence function C ; and permitting execution of the consequence only if all elements of $\{C(e)\}$ are equal, otherwise withholding execution.” Dependent claims: e.g. “if $|\{C(e)\}| > 1$, compute a minimum-cost next observation to refine E ” or “emit a receipt ‘UNNECESSARY_WORLD_RESOLUTION’ when $|\{C(e)\}| = 1$ but world identity is unresolved.” Figures: a flowchart like above.

6. Observation Optimization Engines

Mechanism: These engines support the above consequence gating by determining how to minimize information use. Two complementary problems:

- **Minimum Sufficient Observation:** Given admitted evidence X , find a subset $X_C^* \subseteq X$ of minimal size that yields the same consequence determinacy (i.e. collapsing unnecessary distinctions). This identifies which observations were redundant.
- **Minimum Missing Observation:** If the consequence is indeterminate, find the cheapest additional observation q that (potentially) resolves the outcome ($V_C(q) - \text{Cost}(q)$ is maximized). This is essentially a *value-of-information* decision rule. Include a “no-op” choice ($q=\emptyset$) representing “do not observe further” if cost outweighs benefit.

Pipeline:

```
flowchart TD
    Start[Given evidence set X] --> Check{Consequence determinacy?}
    Check -- Yes --> F1[Compute  $X_C^*$  (min sufficient)]
    Check -- No --> F2[Find  $q^*$  maximizing  $V(q) - \text{Cost}(q)$  or DO_NOT_OBSERVE]
    F1 & F2 --> End
```

Inputs/Outputs: Input is current evidence set and outcome function. Outputs: either a reduced evidence subset, or a chosen query (or decision to halt queries).

Data Structures: Represent evidence as a set or vector; cost and utility functions (could be numeric). Possibly a dependency graph to test effect of removing evidence.

Implementation Evidence: Not implemented. Mentioned only in discussion (“MinimumSufficientObservation”, “MinimumMissingObservation”).

Claimable Features:

- *Observation Abstraction:* Mechanism that given evidence X and a consequence, computes a minimal sub-evidence sufficing the decision.
- *Value-of-Info Decision with Null Option:* Selecting q^* from candidate observations plus the null observation to maximize net gain.
- *Information-Resolution Policy:* Formalizing that if no low-cost observation suffices, the system can output a decision to halt (withhold) without infinite probing.

Prior Art: The concept of minimal sufficient statistics in POMDPs and sensor management is well studied. TargetSpace explicitly uses “minimum sufficient observation” [10†L61-L69] as a research goal. Value-of-Information is classic in decision analysis [10†L63-L69]. However, linking these into an execution gate is novel. We will review work on active sensing, “probe selection” in diagnosis, and POMDP policy planning. A

search for “value of information monitoring” yields mostly AI/ML papers, not patents. We should check if any patent covers “adaptive sensing for operation gating”.

Novelty/Obviousness: As standalone algorithms, minimal evidence selection can be seen as abstract. Yet in context, it's part of an execution control machine. We should tie the claim to the gating context. To bolster, an implementation example (e.g. simulation or chart of cost vs. information) could be done.

Provisional Filing: Likely rolled into the Consequence gate claims (as a dependent feature). For completeness, we may claim: *“In the above method, when multiple outcomes exist, selecting an additional evidence query q that maximizes (value – cost), where value = reduction of outcome entropy.”** Figures: maybe a decision tree of queries.

7. Independence Challenge Machine (Derivation Admission)

Mechanism: This family underlies the *Derivation Admission* gate. Given a basis **B** (justifications) and a prohibited factor **L** (e.g. a sensitive label), the system tests whether the basis is truly independent of **L**. It does so by generating synthetic counterfactuals (e.g. swap labels, permute values, or delete **L**) and re-evaluating the consequence **C**. The result is a receipt cataloging which challenge types (swap, deletion, etc.) were “survived” (still no change in C) or “failed”. A label of **INDEPENDENT_SURVIVED** or **INDEPENDENT_VIOLATED** is issued.

Inputs/Outputs: Input: evidence **B**, prohibited source **L**, and a partition of possible interventions (e.g. label permutations). Output: an **IndependenceReceipt** listing “survived” challenges (e.g. LABEL_SWAP) and failures.

Claimable Features:

- *Enumeration of Independence Challenges:* Systematically testing multiple interventions on a basis to assess independence.
- *Challenge-Scoped Proof:* Emitting a structured receipt detailing which categories of intervention were tested and their outcomes.
- *Limited Theorem Scope:* Not claiming full independence, only within enumerated challenges.

Prior Art: This is closely aligned with meridian’s concept of “Action Acceptance” and independence (TSIL and others do label swaps as examples). No standard patents likely; it’s quite specific. Partial mention in the conversation (the receipts say NOT_ESTABLISHED for semantic equivalence, which is akin to “we did bounded tests only”).

Novelty: The idea of enumerating and recording multiple interference tests is unusual. Check “information flow control” patents (noninterference), though those are often theoretical.

Note: We group this under Derivation Admission, as it guards *how* the basis was produced, ensuring the consequence isn’t tainted.

Comparison of Patent Families

Family	Strength	Status (impl.)	Key Novelty	Prior-Art Risk	Next Work (Enablement)
A. Property-Grant Validation Runtime	★★★★★	Partially (per-property checks visible)	Independent per-property validators & receipts 【28†L50-L58】 【29†L8-L11】	Medium (similar to ABAC)	Prototype proof-of-concept for property grants; secure receipts; performance.
B. Consequence Execution Gate (Quotient)	★★★★★	Planned (Identifiability code exists)	Execution gating by consequence equivalence; UNNECESSARY_RES 【23†L103-L111】 【34†L164-L166】	Low (novel combination)	Implement dispatch gate + optimization; test distinguishable outcomes.
C. Runtime State Admission	★★★★☆	Partially (basic checks done)	Typed failure predicates, replay re-admission	Medium (admission control is known)	Complete predicate suite, replay engine, receipts.
D. Distinction Disposition Engine	★★★★☆	Conceptual (no code)	One-to-one lawful disposition for each distinction	Low (unique idea)	Define algorithms for each disposition type; example scenarios.
E. Observation Optimization	★★★★☆	Not implemented (concept only)	Minimally sufficient evidence and do-not-observe decisions	Medium (decision theory)	Formalize V(q)-Cost computations; integrate with gate.
F. Receipt/Provenance Architecture	★★★★☆	Partially (per-distinction receipts)	Per-property & per-distinction receipts for each check 【23†L103-L111】 【28†L50-L58】	Low (receipt ideas exist)	Develop receipt schema (e.g. PROV); implement generation.

Family	Strength	Status (impl.)	Key Novelty	Prior-Art Risk	Next Work (Enablement)
G. Independence Challenge Generator	★★★★☆☆	Not implemented (discussed)	Enumerated causality attacks on evidence basis	Medium (noninterference known)	Prototype independence tests and receipts.

Each family's **strength** rating reflects a combination of novelty, feasibility, and alignment with the project's core. For example, the *Property-Grant Runtime* (A) is now top priority: it is concrete (each validator is a function) and yields clear runtime behavior. The *Consequence Gate* (B) is equally strong as it embodies the project's governing principle ("no unearned consequence"). We have grouped some features (like *false-withholding*) into B. Families C–F are also strong candidates. The main enablement gap across all is to build and document working code/prototypes that clearly execute these mechanisms, so that claims can be supported by actual examples.

References

- W3C PROV Data Model – defines entities/activities for provenance [28†L50-L58] .
- *TargetSpace* "Minimum Sufficient Observation" – defines minimal evidence preserving outcomes [10†L61-L69] .
- *In-toto supply-chain framework* – example of stepwise signed receipts [29†L8-L11] .
- Runtime verification surveys – "Modern techniques address partial observability..." [13†L69-L73] .
- Meridian Verity TSIL patent brief – "receipt-gated, fail-closed admission" [23†L103-L111] [34†L164-L166] .
- Others as discussed (patents.justia, academic papers on POMDPs and value-of-information) for background context.